

数学争鸣

人人都能编写程序

Claire Tristram

软件越来越不能承受它自身的复杂性。什么是 Charles Simonyi 的解决方案？他要让编程工具变得那么简单，以致于非专业编程人员也能使用它。

很少有软件专家能像 Charles Simonyi 对计算的发展产生如此革命性的影响，也很少有软件专家像他那样由于自己的努力而得到如此丰厚的回报。作为 Xerox 公司设在 Palo Alto¹⁾ 的研究中心的一名科学家，Simonyi 于 20 世纪 70 年代发明了第一个文字处理软件 Bravo，它在屏幕上准确显示出文件打印的效果，即通常所说的“所见即所得”。后来，Simonyi 加入了微软 (Microsoft)。那还是微软创办初期，只有三十几个人。在微软，他是公司的主要设计者，领导了 Word 和 Excel 的研发。在这个过程中，他也成了亿万富翁，《福布斯》(Forbes) 杂志最近列出的美国最富有的人中，他位居第 209。

去年，Simonyi 突然离开了他在微软 21 年的任职，开办了自己的意图软件公司 (Intentional Software)。到目前为止，这个公司全部由他本人投资。从某种意义讲，意图软件是个不大的公司，目标也并不宏伟；Simonyi 不指望有一个产品卖它几年。但从另一种意义讲，他的目标极为宏伟，甚至“雄心”二字都不足以描述所需付出的努力。简单地讲，Simonyi 要把软件从它自身的复杂性中解放出来，使得真正的计算机技术的潜能最终得以发挥。

Simonyi 深知客居他乡所意味的：他 17 岁时离开匈牙利到海外求学——非法的，然后就再也没回去。虽然他仍略带口音，但他不择词令所表达的看法毫不含糊，尤其说到他最感兴趣的话题。

他说，“我们一直在关注软件领域不断改进的过程，然而这种改进的效果并不明显，因为它永远不能解决人类自身不完善带来的问题。”

Simonyi 则另辟蹊径，在如何写软件、甚至在如何认识软件方面提出了一个革命性的改变的思路。他说：“人们通常所做的改进工作已使人们忘记了软件到底是怎么回事。”

为何软件需要一个革命性改变？因为今天软件技术已面临危机，它的复杂性已超出了我们的驾驭能力。当一个程序超过几百行，你就几乎不明白这个软件是怎么回事了——而如今台式电脑的软件包含几百万条代码。我们不理解就不可能改正程序的错误：因此 2000 年在美国有 25% 的商业软件项目被取消，这意味着仅此一项造成美国当年经济损失达 600 亿美元 (见“为何软件如此之坏”，MIT Technology Review, 2002, 7 月 / 8 月)。

尽管软件已难以承受它自身的复杂性，我们却还没开始去探索如何解决。Simonyi

原題：Everyone's a Programmer. 译自：MIT Technology Review, Nov. 3, 2003.

1) Palo Alto, 位于美国加州“硅谷”(silicon valley), 著名的 Stanford 大学的所在地。——校注

认为，挑战的关键在于要找到一种编程方法使编程人员和用户都能读懂。Simonyi 的解决方案是什么呢？要发明一种既简单、功能又很强的编程工具，使得软件好像由软件自身写成，就像 Excel 软件那样自动添加数字栏目，或象 Word 软件自动把文件格式化。

Simonyi 主张：“软件要像 PowerPoint 演示那样容易编辑”。也就是说，要给它一个同样直观的界面。

Simonyi 企图解决软件开发的一个最根本的问题，即“让代码看起来象设计”。如果他获得成功，用户就能做出高水平的设计并让程序实现。这种设计更像流程图，而不是一行行的代码。程序代码是从设计自动生成的。

作为第一个“所见即所得”界面的发明者，Simonyi 似乎是唯一能迎接挑战的人。但这的确是很难的。有人说这是不可能实现的：从编写软件以来人们就一直在试图创建代表性的模型来自动生成那些复杂的代码，迄今为止他们只取得非常初步的成功。

也有人说我们没有其它选择。确实，IBM, Sun Microsystems 以及另外一些企业和科研机构也在做着类似的工作，使得编程简单化和自动化。

Grady Booch 是 Simonyi 的朋友，统一模型语言 (Unified Modeling Language 或 UML) 的发明者之一。与 Simonyi 提出的方案类似，这个软件语言能让编程人员只需考虑软件做什么，而不用管真正代码的细节，不用管它是采用 Java, C++ 还是其它今天使用的语言。去年 (2002 年)12 月，IBM 花了 21 亿美元收购了理性软件公司 (Rational Software)，就在这个公司 Booch 和他的同事们开发了统一模型语言。Booch 说：“我们正在解决从来没有过的更为复杂的软件问题。”

Booch 引纳米微处理器这样的新技术为例 (这种新技术将使计算机更为强大，只要我们能找到有效的方法为这种计算机编写软件) 指出，“如果看一下我们将会走到哪里，那是永无止境的”。换句话说，软件必须赶上硬件。而今天我们编写软件的方式是做不到的。

让意图重见天日

我们如何使代码看起来像设计？Simonyi 主张，首先要了解目前编程实践中的问题是什么。

他说“目前，编程正好和开采钻石过程相反，在开采钻石时你挖出大量的泥土，而找出一点点昂贵的钻石。可编程时，你从一个有价值的真正意图开始，随后却把这种意图埋在泥土中”。

依 Simonyi 的观点，软件开发者是注定要失败的，因为他们同时要做 3 件事情——而其中仅有一件事适合他们。首先，他们要理解用户复杂的需求，他们可能来自保险专家，会计师，或飞机设计师，软件是为他们做的。编程人员必须尽可能吸收用户多年积累掌握的专业知识。做到这一点即使不是完全不可能，对他们的专长来说也是不合适的。

其次，编程人员要把客户的需求翻译成计算机系统能理解的算法和界面。Simonyi 认为这是编程人员的核心任务。但目前这项任务完成得很差：软件完成后，用户无法修

改，甚至无法理解软件如何反映了他们的意见。

第 3. 为使计算机正确执行命令，编程人员必须写出与机器一样完美无缺的精确代码。尽管软件公司都在宣称他们的软件开发过程优于他们的竞争对手，事实上没有错误的编程是不可能的，因为人不是机器。最近（美国）卡内基·梅隆大学的一项研究发现，编程人员编写的程序平均每 1000 行代码就有 100 至 150 个错误。

Simonyi 希望为编程人员解脱上述第 3 项工作的全部及第 1、2 项工作的大部负担。他设想的不只使像工蜂那样的编程工作实现自动化，而且要让编程界面如此直观，以至于保险专家、会计师，或飞机设计师都能看到他们的作用，并能借助他们自己的专业知识改进程序，而无需通过程序编写人员的介入。一旦编程人员从那些压在他们身上的不适当的任务中解放出来，他们就可以集中精力完成他们被唯一训练的事情，即程序设计。

“真正的问题是，我们要用这个软件去做什么？”Simonyi 说，“这正是意图编程使我们有可能集中精力去做的事情。当你要创建一个非常好的卫生保健系统时，你就应该专注于卫生保健问题，并考虑如何解决这些问题。但按我们现在编写程序的办法，却顾不上去理解问题本身，因为编程人员更关心如何把数字分类，如何把数据存盘。”

用图编写程序

如果你让 Simonyi 解释怎样使像工蜂那样的编程工作可以自动完成，从而消除由于人为差错造成的程序错误，他会给你举出喷气发动机的例子。

“拿涡轮叶片来说，”他说，“它们必须做得非常精确。即使由很细心的熟练工人加工叶片，仍然不可能达到你要求的精度，而必须另造一台机器来加工叶片。其中会有人干预这个过程吗？当然，制造、维修和调整机器必须由人来完成。机器会出错吗？一定的。机器一旦出错会很严重，你能马上发现，并改正它。程序编码也是如此。不需要人去接触编码。否则程序易于出错。人能做些什么？是的，他们能制造这种机器。”

机器来编写软件会是什么样呢？这种机器本身也是软件。但它的功能不是解决最终问题——执行某种新的家庭或办公室任务。说得更恰当一些，它会是一个软件“生成器”，由它来编写特定的一段软件。告诉生成器写什么程序，通过一个容易理解的界面，有时叫做“模型语言”来完成。

目前最广泛采用的模型语言是统一模型语言。它是由 Booch 和他的编程员同事 James Rumbaugh 和 Ivar Jacobsen 在理性软件公司做的工作开发出来的。目前，开放源软件群体正在对它开发，其中包括 IBM 和称为目标管理集团 (Object Management Group) 中的工业联合体的其它成员单位。统一模型语言是一种建造图表的系统。它的意图是要使大型软件课题的管理人员能看到他们的设计，在程序员坐下来用程序语言 Java 或 C++ 编写程序前确保设计能满足用户的要求。

Simonyi 的观点，也是启动他的公司的动机，在于提出模型的概念，并进一步把模型和编程紧紧连在一起，最终合而为一。程序员和用户可以从许多不同的角度察看他们所创造的模型，只需修改模型就可随意修改程序。这有点像建筑师只要画出蓝图，就象魔

术师那样产生蓝图中的结构，如果修改蓝图也能自动重建结构。

与 Simonyi 共同创办意图软件公司的 Gregor Kiczales 说，它其实就是原来的“所见即所得”的概念。去年在创办意图软件公司之前他离开了在 Polo Alto 研究中心的兼职位置，但后来他又回到在温哥华的 British Columbia 大学的永久位置当他的计算机科学教授。Kiczales 赞成被称为面向多方面的程序设计方法 (aspect-oriented programming)，它能让程序员即使在很复杂的程序中，也能迅速编辑所有场合的相关命令，就像文字处理程序能找到和替代每一个拼错的单词。这是一种支持意图编程的基础技术。Kiczales 说，意图编程的思想，已用于文字处理，是否对其他程序也这么做？目前他仍是意图软件公司的顾问。

Simonyi 描述生成程序的新模型时说，它看起来很像 PowerPoint 的调色板，任何人都可用它制作幻灯片，把文本、图表或图像粘贴到一个直观的但又是虚拟的工作空间的不同位置。但是意图软件公司深信不同形式的问题可能采用不同的界面以满足不同用户的需要。例如，从事计算机模拟的科学家可能要利用程序生成器让机器把数学表述转换为相应的代码。

Simonyi 所想象的那种软件不仅不再要求未来的程序员像机器那样干活，而且可使特定领域——保险，会计，卫生保健——的专家十分方便地修改他们的软件，无需不断地去麻烦程序员。如果你是一个公司会计，使用常用的财务软件，当一个新的税法出来，Simonyi 断定，“你自己就可以编辑你想做的事情的说明，然后再运行生成器，你并不需要打搅程序员。生成器以计算机的速度运行，比人要快约 10 亿倍，而精确性提高 10 亿倍，这样你就完成了你需要的改变。”

软件的再生

即使一直很乐观的 Simonyi 表示，从这些工具的开发，到他的公司把产品投入市场至少需要两年。这些工具给编程所带来的简易并非唯一的好处，还使程序员创建的软件所能达到的复杂程度用现在的方法是无法达到的。

关于如何采用模型来拯救软件增加的复杂性，Simonyi 的同事和竞争对手有非常不同的想法。例如 Booch 希望通过建立覆盖面更宽的 Kiczales 的“面向多方面的程序设计”的版本，充实统一模型语言，使得例如像安全和认证的功能自动覆盖到整个软件系统。他还致力于“找到方法来表达越来越强的抽象能力”——或者说，超越单个程序的模型，达到巨型软件项目的总体结构模型。

“编写软件本质上是一个非常复杂的问题，而且越来越糟糕。”Booch 说，“我们刻板地书写应用软件，现在它们的代码已有几百万行。为此，你必须向简单化方面思考，模型能帮助我们。”

James Gosling 是 Sun 实验室的一名研究人员，Java 语言的发明人。他是另一个近期支持这种模型的人。但是他研究的重点不同。认识到软件业常常重新用老的程序，他建议找到一种方法把现有的程序插入一种图解程序的模型工具，

(下转 11 页)

Drinfeld 为 Fields 奖得主之一, 其获奖成就的一部分是关于 Langlands 猜想方面的工作.

在 1994 年 Zurich 大会上, A. Wiles 作了题为“模形式, 椭圆曲线和 Fermat 大定理”的报告. 他解释他证明 Fermat 大定理的方法. 在大会期间, 他的证明中有一处仍不完善, 但在几周之后 (与 R. Taylor 合作) 完成了证明. 在 1994 年大会文集中已经提到他们即将发表的两篇证明文章.

3.13 最后评论

从 1900 年大会 Hilbert 的报告到 1936 年大会, 代数数论在 ICM 中未得到充分反映. 这有两个原因: 1. 在 1930 年以前, 中欧和日本以外的人很少知道代数数论, 而大会邀请程序由主办国的组委会决定. 2. 第一次世界大战的战败国奥地利、保加利亚、德国和匈牙利在 1920 年和 1924 年大会上未被邀请. 在德国数学家缺席的情形下, 代数数论有影响的代表为 L. E. Dickson 和 R. Fueter, 但是他们对于发展这一理论所起的作用不大.

从 1936 年开始, 代数数论在大会中有适当的反映. 而从 1962 年起这种反映就相当充分和令人满意了.

参考文献 (略)

(冯克勤 译 冯绪宁 校)

(上接 81 页)

而不是要求程序员盯着数以几百万行的文本. 理论上我们能对所有老的没有错误的程序提供这样的工具, 考察它们所表达的设计, 识别它们逻辑上的缺陷.

Gosling 说, 像大多数程序员一样, 我喜欢从头做起. 但人们几乎从来没有那种奢望. 我们正在创建一种工具, 它能读很大的系统, 例如 2—3 百万行, 并能使它易于理解.

以代码命名的课题 Jackpot 还很小, 不在 Sun 实验室的课题开发计划之内. 但它是 Booch 和 Simonyi 所热衷的, 因为它是模型正在成为推动产业的一股力量的又一个标志.

什么时候我们能只从模型单独生成复杂的代码? 虽然意图软件公司还没有投入市场的产品, 它计划在 2004 年具体发布. IBM 购入理性软件公司, 并承诺统一模型语言的研发, 以及 Sun 实验室对此概念的兴趣, 这些都表明这些思想正在达到关键阶段.

这些预言家所面临的挑战是巨大的. 但是他们的成功也是必然的. 因为他们所建议的事情的意义, 远远超出那些相信我们能从几百万手工制作的程序段 (任何一个程序段的一个小缺陷都会毁掉整个程序) 做成一件完美的东西. 这正是我们企图对今天的软件要做的.

Simonyi 说, 软件是那样灵活, 期望是那么宏大, 和我们明天将要实现的目标相比, 我们今天已习惯的普通改善是微乎其微的. 瞧, 硬件研发人员努力按 Moore (摩尔) 定律所做到的, 现在该轮到软件了.

(吴邦贤 林继玲 译 陆柱家 校)